

Skill 12:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_COMMANDS 5

/* =====
   Execute a single pipe
   Example: ls | wc
   ===== */

void single_pipe()
{
    int pipefd[2];
    pid_t p1, p2;

    printf("\n=====\\n");
    printf("SINGLE PIPE\\n");
    printf("=====\\n");

    /* Create pipe */

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return;
    }

    /* First process */

    p1 = fork();

    if (p1 == -1)
    {
        perror("fork");
    }
}
```

Multiple Pipes

Example:

```
ls | sort | head | wc
```

```
===== */
```

```
void multiple_pipes()
{
    char *commands[MAX_COMMANDS] =
    {
        "ls",
        "sort",
        "head",
        "wc",
        "cat"
    };

    int command_count = 4;

    int pipes[MAX_COMMANDS - 1][2];

    pid_t pids[MAX_COMMANDS];

    printf("\n=====\\n");
    printf("MULTIPLE PIPELINE\\n");
    printf("=====\\n");

    printf("Pipeline:\\n");

    for (int i = 0; i < command_count; i++)
    {
        printf("%s", commands[i]);

        if (i < command_count - 1)
            printf(" | ");
    }

    printf("\\n\\n");
}
```

```

for (int i = 0; i < command_count; i++)
{
    pids[i] = fork();

    if (pids[i] == -1)
    {
        perror("fork");
        return;
    }

    if (pids[i] == 0)
    {
        /* =====
        Connect input from previous pipe
        ===== */

        if (i > 0)
        {
            dup2(pipes[i - 1][0], STDIN_FILENO);
        }

        /* =====
        Connect output to next pipe
        ===== */

        if (i < command_count - 1)
        {
            dup2(pipes[i][1], STDOUT_FILENO);
        }

        /* =====
        Close all pipe descriptors
        ===== */
    }
}

```

```

    }
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* =====
Wait for all processes
===== */

for (int i = 0; i < command_count; i++)
{
    waitpid(pids[i], NULL, 0);
}

printf("\nLong pipeline execution completed.\n");
}

/* =====
MAIN
===== */

int main()
{
    printf("=====\n");
    printf("PIPE AND MULTIPLE PIPE PRACTICAL\n");
    printf("=====\n");

    /* Test single pipe */
    single_pipe();

    /* Test multiple pipes */
    | multiple_pipes();

```

```

bhanu@DESKTOP-DE32HQ8:~$ cd ~
mkdir -p ~/pipes_practical
cd ~/pipes_practical
bhanu@DESKTOP-DE32HQ8:~/pipes_practical$ pwd
/home/bhanu/pipes_practical
bhanu@DESKTOP-DE32HQ8:~/pipes_practical$ nano pipes.c
bhanu@DESKTOP-DE32HQ8:~/pipes_practical$ gcc pipes.c -o pipes
bhanu@DESKTOP-DE32HQ8:~/pipes_practical$ ls
pipes  pipes.c
bhanu@DESKTOP-DE32HQ8:~/pipes_practical$ ./pipes
=====
PIPE AND MULTIPLE PIPE PRACTICAL
=====

=====
SINGLE PIPE
=====
2
Single pipe execution completed.

=====
MULTIPLE PIPELINE
=====
Pipeline:
ls | sort | head | wc

      2      2      14

Long pipeline execution completed.

=====
PRACTICAL COMPLETED
=====
bhanu@DESKTOP-DE32HQ8:~/pipes_practical$ |

```